

ALLUCI SOVEREIGN AGENT

UI/UX FEATURE GAP ANALYSIS · WITH DEVELOPER SPECIFICATIONS

Identifies every UI feature, function, and capability present in OpenClaw that is missing from Alluci.

For each gap: what the UI needs, what the back-end must implement, and reference code.
Aesthetic / visual design intentionally excluded — functionality only.

1	Chat Interface — Compose, Thread & Session Controls	7
2	Message Queue, Streaming & Context Indicators	5
3	Session Management Panel	6
4	Token Usage & Cost Analytics Dashboard	9
5	Scheduled Automation (Cron Engine)	9
6	Channel / Bridge Management	10
7	Agent Configuration & File Editor	8
8	Skill Registry & Plugin Lifecycle	6
9	Node & Device Management	5
10	Config Editor (Schema-Driven, Live)	4
11	Log Streaming & Observability	4
12	Debug / RPC Console	4
13	Navigation, Shell & Global Controls	5
TOTAL		82

How to Read This Document

Each gap is structured in four parts:

- Gap description — what OpenClaw has that Alluci is missing.
- Front-end changes — React/TSX changes needed in the Alluci UI to surface the feature.
- Back-end required — Python/FastAPI endpoints, database schema, or WebSocket events the front-end depends on.
- Reference implementation — key lines of TypeScript or Python to make it concrete.

Scope note

All gaps are purely functional — no design tokens, colour values, animation classes or layout aesthetics are mentioned. Every item describes a missing capability, control, data surface, or workflow, not a visual style.

Gap	A feature/control/flow present in OpenClaw but absent from Alluci
Stub	Code that exists in Alluci but does nothing (empty function body, unconditional return)
Missing	Feature has no corresponding code in Alluci at all
WS event	A WebSocket push event the server sends without a client request
RPC method	A request-response call over the WebSocket or REST layer
Store slice	A Zustand state property + setter that needs to be added to useStore.ts

Compose bar, thread controls, session selector, focus mode & sidebar

1.1 Session Selector in Chat Header

OpenClaw’s chat header contains a session-key picker. The user can switch between named sessions (e.g. 'main', 'research', 'coding') from a dropdown directly in the chat view without leaving the screen. Alluci has no such control — the session key is fixed to whatever was set in the API config and cannot be changed without editing environment variables.

 FRONT-END CHANGES • Add a <code><select></code> or custom dropdown in the chat header (next to the model badge) populated with sessions from the store. • On change, dispatch <code>setActiveSession(key)</code> which updates <code>VITE_SESSION_KEY</code> and re-initialises the connection. • Show the current session key as a monospace label when the selector is collapsed. • Add 'New Session' as the last option in the list, invoking <code>handleNewSession()</code>.	 BACK-END REQUIRED • GET <code>/api/sessions</code> → returns <code>[[key, label, updatedAt, totalTokens]]</code> • POST <code>/api/sessions {key, label}</code> → creates a new session key entry • The session key is passed as a header (X-Session-Key) on every LLM request. • Add a sessions table: key TEXT PK, label TEXT, created_at, updated_at.
 REFERENCE IMPLEMENTATION <pre data-bbox="215 621 1347 919"> // useStore.ts - add slice sessions: SessionRow[], activeSessionKey: string, setActiveSessionKey: (key: string) => void, // ChatHeader.tsx <select value={activeSessionKey} onChange={e => setActiveSessionKey(e.target.value)}> {sessions.map(s => <option key={s.key} value={s.key}>{s.label} s.key</option>)} <option value='__new__'>+ New Session</option> </select> </pre>	

1.2 Model Badge & Thinking Level Toggle

OpenClaw shows the active model name as a badge in the chat header, plus a 'Thinking' toggle button that cycles through off / low / medium / high. The thinking level drives extended reasoning in Claude models. Alluci's chat header shows only the static view title — no model indicator and no thinking control.

 FRONT-END CHANGES • Add a model badge next to the session selector, showing the short model name (e.g. 'claude-3.7'). • Add a Thinking toggle button (icon + label) that cycles through a configurable set of levels. • Store <code>showThinking</code>: boolean and <code>thinkingLevel</code>: string in <code>useStore</code>. • When thinking is on, wrap outbound messages with the appropriate API parameter.	 BACK-END REQUIRED • GET <code>/api/config</code> → include <code>active_model</code> and <code>supported_thinking_levels</code> • POST <code>/api/session-config {sessionKey, thinkingLevel}</code> → persists per-session override • In the LLM call, pass <code>thinking: {type:'enabled', budget_tokens: levelToTokens(level)}</code> when thinking is active.
 REFERENCE IMPLEMENTATION <pre data-bbox="215 1640 1252 1782"> // sovereignService.ts - augment sendMessage const params = thinkingLevel !== 'off' ? { thinking: { type: 'enabled', budget_tokens: LEVEL_MAP[thinkingLevel] } } : {}; await llmClient.complete({ ...baseParams, ...params }); </pre>	

1.3 Focus Mode (Nav-Collapsed Full-Screen Chat)

OpenClaw has a focus mode button in the chat header that collapses the navigation sidebar and maximises the chat thread to full window width. A second click restores normal layout. Alluci has a sidebar collapse toggle but no dedicated focus mode button inside the chat view.



FRONT-END CHANGES

- Add a focus-mode icon button to the CommandBar or chat header area.
- On click, dispatch `setSidebarCollapsed(true)` and set a new store flag `isFocusMode: true`.
- When `isFocusMode`, also hide the topbar view title and any secondary chrome.
- Second click or Escape key restores normal layout.



BACK-END REQUIRED

- No back-end changes required — this is purely client-side layout state.



REFERENCE IMPLEMENTATION

```
// useStore.ts
isFocusMode: false,
toggleFocusMode: () => set(s => ({
  isFocusMode: !s.isFocusMode,
  isSidebarCollapsed: !s.isFocusMode || s.isSidebarCollapsed
})),
```

1.4 Abort / Stop Generation Button

OpenClaw shows a Stop button while a streaming response is in progress. Clicking it sends an abort signal to the gateway, immediately halting the token stream and preserving the partial response. Alluci's CommandBar shows a spinner while `isProcessing` is true but provides no abort control — the user must wait for the full response.



FRONT-END CHANGES

- In CommandBar, replace the submit button with a Stop button when `isProcessing` is true.
- The Stop button calls `handleAbort()` which closes the current request and sets `isProcessing = false`.
- Mark the partial transcription with a `[stopped]` suffix or a visual indicator.



BACK-END REQUIRED

- `POST /api/chat/abort {sessionKey}` → cancels the active inference stream for that session.
- Backend uses an `AbortController` per session; calling this endpoint triggers `controller.abort()`.
- The streaming response endpoint should catch `AbortError` and flush a partial result.



REFERENCE IMPLEMENTATION

```
// useInteractions.ts
const abortRef = useRef<AbortController | null>(null);
const handleAbort = () => { abortRef.current?.abort(); setIsProcessing(false); };

// In handleCommandSubmit:
abortRef.current = new AbortController();
const res = await fetch('/api/chat', { signal: abortRef.current.signal, ... });
```

1.5 Markdown Artifact Split-Pane Sidebar

When the assistant returns a Markdown artifact (a fenced code block or a specially-tagged section), OpenClaw opens a resizable side panel that renders the artifact while keeping the chat thread visible. The divider is draggable and the ratio persists in `localStorage`. Alluci renders all output inline in the terminal view with no sidebar.



FRONT-END CHANGES

- Add a `sidebarContent: string | null` to `useStore`. When non-null, render a split-pane layout.
- The left pane is the existing `TerminalView`; the right pane is a Markdown renderer (`react-markdown`).
- Add a draggable `ResizableDivider` component between panes that updates `splitRatio` in the store.
- Detect artifact markers in assistant messages: code blocks with `lang='artifact'` or wrapped in `<artifact>` tags.
- Add a close button on the sidebar pane that sets `sidebarContent` to null.

⚙️ BACK-END REQUIRED

- No new endpoints required.
- The daemon can annotate responses with a `X-Has-Artifact: true` header or inline JSON marker.
- Alternatively, the front-end can heuristically detect multi-line code blocks > 20 lines and auto-open.



REFERENCE IMPLEMENTATION

```
// App.tsx - conditional layout
const {sidebarContent} = useStore();
return sidebarContent ? (
  <div style={{display:'flex',flex:1}}>
    <div style={{flex: splitRatio}}><TerminalView /></div>
    <ResizableDivider onResize={setSplitRatio} />
    <div style={{flex: 1-splitRatio}}><MarkdownPanel content={sidebarContent} /></div>
  </div>
) : <TerminalView />;
```

1.6 'Jump to New Messages' Button

When the user scrolls up in the chat thread and a new message arrives, OpenClaw shows a floating 'New messages ↓' pill at the bottom of the thread. Clicking it scrolls to the latest message. Alluci auto-scrolls on every new message regardless of scroll position, overriding the user's position.



FRONT-END CHANGES

- Track scroll position in the TerminalView scroll container with an onScroll handler.
- Add showNewMessages: boolean to store, set true when a new transcription arrives and the user is not at the bottom.
- Render a fixed-position 'Jump to bottom' button in the TerminalView when showNewMessages is true.
- On click (or when user scrolls to bottom), set showNewMessages = false and scroll to the bottom.
- Only auto-scroll when the user is already at the bottom (within 100px of messagesEndRef).

⚙️ BACK-END REQUIRED

- No back-end changes required.



REFERENCE IMPLEMENTATION

```
const isAtBottom = () => {
  const el = scrollContainerRef.current;
  return el && (el.scrollHeight - el.scrollTop - el.clientHeight < 100);
};

useEffect(() => {
  if (isAtBottom()) messagesEndRef.current?.scrollIntoView({behavior: 'smooth'});
  else setShowNewMessages(true);
}, [transcriptions]);
```

1.7 Message Copy Button (Per-Message)

OpenClaw renders a copy-to-clipboard button on each assistant message bubble on hover. Alluci already has a copy button per message but it renders as static text 'COPY' with no icon and is visible at all times, not just on hover. The gap is that the copy control in Alluci is always visible and styled as plain text rather than being a discreet icon-only hover affordance. While minor, this is a distinct functional pattern — on hover only, icon-based — that is missing.



FRONT-END CHANGES

- In TerminalView, add an isHovered: boolean state per message (useRef or CSS :hover).
- Show the copy button only when the message is hovered (opacity transition from 0 to 1).
- Replace the plain text 'COPY' label with a Lucide Clipboard icon.
- Add a 'Copied!' tooltip that auto-hides after 1.5 s using a setTimeout.

⚙️ BACK-END REQUIRED

- No back-end changes required.

Compaction dividers, fallback banners, queue, reading animation, source chips

2.1 Context Compaction In-Thread Divider

When OpenClaw compacts the context window (summarises old messages to stay under the token limit), it inserts a visible horizontal divider in the message thread with a label like 'Context compacted'. This gives the user a clear visual record of when the conversation history was pruned. Alluci performs compaction silently — no marker appears in the thread.

FRONT-END CHANGES

- Add a compaction marker message type to the Message type: {type:'compaction', timestamp, reason}.
- In TerminalView, when rendering a message of type 'compaction', render a full-width horizontal rule with a label instead of a chat bubble.
- Add compactionStatus: {active:boolean, completedAt:number|null} to the store.
- Show a toast/banner above the CommandBar while compaction is active ('Compacting context...').

BACK-END REQUIRED

- When the daemon compacts context, append a {type:'compaction', timestamp, summary_length} object to the session's message list.
- Push a WS event: {event:'compaction.status', data:{active:true}} at start and {active:false, completedAt} on completion.
- The compaction event must be distinguishable from regular transcription events.

REFERENCE IMPLEMENTATION

```
// sovereignService.ts — on compaction event
case 'compaction.status':
  setCompactionStatus(data);
  if (!data.active) {
    setTranscriptions(prev => [
      ...prev,
      {type:'compaction', timestamp: new Date().toISOString()}
    ]);
  }
  break;
```

2.2 Model Fallback Banner

When OpenClaw's model router switches to a fallback model (because the primary is unavailable or rate-limited), it shows a dismissible amber banner above the compose area: 'Fallback active: claude-3-haiku'. The banner auto-dismisses after 8 seconds. The phase also shows 'Fallback cleared' when the primary comes back. Alluci performs silent fallover — the user never knows the model changed.

 FRONT-END CHANGES • Add fallbackStatus: {active: boolean, primary: string, current: string, reason: string} null to the store. • Render a dismissible banner strip between the message thread and the CommandBar when fallbackStatus is non-null. • Auto-dismiss after 8000ms using a useEffect with setTimeout.	 BACK-END REQUIRED • ModelRouter must emit a WS event when it switches models: {event:'model.fallback', data: {primary, active, reason}}. • Emit a second event when the primary is restored: {event:'model.fallback.cleared'}. • These events can be delivered via the existing SSE or WebSocket stream.
 REFERENCE IMPLEMENTATION <pre data-bbox="215 556 1011 793">// In model_router.py async def select_model(session_key, preferred): for model in [preferred] + fallbacks: if await self.is_available(model): if model != preferred: await self.emit_event('model.fallback', {'primary': preferred, 'active': model}) return model</pre>	

2.3 Message Send Queue (Queue While Streaming)

OpenClaw queues messages typed while a streaming response is in progress. When the stream completes, the queued message is automatically sent. Alluci's CommandBar disables the submit button (opacity 0.5) during isProcessing, so any typed message must be manually re-submitted after the response finishes. No queuing exists.

 FRONT-END CHANGES • Add messageQueue: string[] to the store. • In handleCommandSubmit, if isProcessing, push the message text to the queue instead of sending immediately. • Show a queue indicator ('1 message queued') above the CommandBar when the queue is non-empty. • In the response-completion handler, after setting isProcessing=false, dequeue and send the next item.	 BACK-END REQUIRED • No back-end changes required — the queue is a client-side buffering mechanism.
--	--



REFERENCE IMPLEMENTATION

```
// useInteractions.ts
const handleCommandSubmit = async (e) => {
  e.preventDefault();
  if (isProcessing) {
    setMessageQueue(q => [...q, textInput]);
    setTextInput(''); return;
  }
  await send(textInput);
};
// In completion handler:
const next = messageQueue[0];
if (next) { setMessageQueue(q => q.slice(1)); await send(next); }
```

2.4 Animated Reading Indicator

While OpenClaw is processing a response, it shows an animated '...' reading indicator in the assistant bubble position — three dots that pulse in sequence. Alluci shows 'ALLUCI_CORE_CALCULATING...' as a static text label. The gap is an animated multi-dot indicator in the message position, which is a conventional and widely-understood streaming affordance.



FRONT-END CHANGES

- When isProcessing is true and no streaming text has appeared yet, insert a placeholder message bubble with an animated three-dot indicator.
- Use a CSS keyframe animation (@keyframes dot-bounce) applied to three elements with staggered animation-delay.
- Remove the placeholder once the first streaming token arrives.

⚙️ BACK-END REQUIRED

- No back-end changes required.

2.5 Grounding Source Attribution Chips

Alluci already surfaces t.sources as anchor chips below each message — this is one of Alluci's existing strengths. However, OpenClaw goes further by also showing the source domain (not just the title) and using a chips row that is scrollable horizontally when more than 4 sources are returned. The title truncation in Alluci (20 chars) is also more aggressive than needed.

FRONT-END CHANGES • Increase title truncation from 20 to 40 characters. • Show the source domain (new URL(s.uri).hostname) as a secondary label below the title. • Wrap the sources row in a horizontally-scrollable container (overflow-x: auto, white-space: nowrap) instead of flex-wrap. • Add a 'SOURCES' count badge: '{n} sources' that collapses the list and expands on click when $n > 3$.	BACK-END REQUIRED • No new endpoints required. • The grounding_metadata from Gemini already returns uri + title. Ensure the daemon passes both fields through without truncation.
---	--

	Session list, labels, per-session overrides, token display, delete
--------------------------------------	--

3.1 Dedicated Sessions Panel with Full Table

OpenClaw has a full Sessions tab showing all active session keys in a table with columns: key, label, kind (global/known/unknown), last updated, token count, and per-session override dropdowns for thinking level, verbose level, and reasoning level. Alluci has no sessions panel — the user cannot see what sessions exist, how many tokens they have used, or switch between them.

FRONT-END CHANGES • Add 'sessions' to the ActiveView union type in useStore. • Add a Sessions nav item to the Sidebar NAV_ITEMS array. • Create a SessionsPanel component that fetches and displays the sessions table. • Table columns: Key (monospace), Label (inline-editable), Kind badge, Updated (relative), Tokens, Actions (Open in Chat / Delete). • Inline label edit: clicking the label cell makes it a text input; blur saves via PATCH.	BACK-END REQUIRED • GET /api/sessions?activeMinutes=60&limit=50 → [{key, label, kind, updated_at, total_tokens, total_cost}] • PATCH /api/sessions/{key} {label?, thinkingLevel?, verboseLevel?} → updates session config • DELETE /api/sessions/{key} → removes the session and its history • Add sessions table: key, label, kind (global/known/unknown), created_at, updated_at, total_tokens.
---	--



REFERENCE IMPLEMENTATION

```
// SessionsPanel.tsx (simplified)
const {sessions, activeSessionKey, setActiveSessionKey} = useStore();
return <table>
  {sessions.map(s => <tr key={s.key}>
    <td><code>{s.key}</code></td>
    <td><EditableLabel value={s.label} onSave={v => patchSession(s.key, {label:v})} /></td>
    <td>{formatTokens(s.total_tokens)}</td>
    <td><button onClick={() => {setActiveSessionKey(s.key); setActiveView('chat')}}>Open</button></td>
    <td><button onClick={() => deleteSession(s.key)}>Delete</button></td>
  </tr>)}
</table>
```

3.2 Per-Session Thinking Level Override

OpenClaw's session table includes a thinking-level dropdown per session that overrides the global default. Levels: off, minimal, low, medium, high, xhigh (or binary off/on for Z.AI-hosted models). Changes are applied immediately via a PATCH RPC call. Alluci has a single global thinking level from the connection setup, not overridable per-session from the UI.



FRONT-END CHANGES

- In the SessionsPanel table, add a <select> column for thinking level.
- Detect the provider from the session config and show binary (off/on) vs multi-level options accordingly.
- On change, call patchSession(key, {thinkingLevel}) immediately.

BACK-END REQUIRED

- PATCH /api/sessions/{key} body: {thinking_level: 'low'|'medium'|'high'|'off'}
- Store thinking_level in the sessions table.
- The LLM call must read the session's thinking_level override and pass it to the model API.

3.3 Per-Session Model Override

OpenClaw allows each session to override the model (e.g. session A uses claude-opus, session B uses claude-haiku). The override is editable from the Sessions tab as an autocomplete text input. Alluci uses the single model from ApiWizard for all sessions.



FRONT-END CHANGES

- Add a model column to the sessions table with an autocomplete text input.
- Populate autocomplete options from the models list fetched from GET /api/models.
- On blur, call patchSession(key, {model}).

BACK-END REQUIRED

- Add a model column to the sessions table in the DB.
- In the LLM request handler, prefer session.model over the global config model when present.
- GET /api/models → [{id, name, provider, context_length}] (used for autocomplete).

3.4 Session Filter Controls

OpenClaw's sessions panel has filter controls: 'Active within N minutes', 'Limit', 'Include global session', 'Include unknown sessions'. These allow the user to narrow the session list. Alluci shows all data without filtering.

 FRONT-END CHANGES • Add a filter row above the sessions table with: a numeric 'Active within' input (minutes), a limit select (20/50/100), and Include Global / Include Unknown checkboxes. • Filters are applied client-side if the session count is small (< 200); server-side otherwise. • Store filter state in local React state (not the global store, since it is panel-local).	 BACK-END REQUIRED • GET /api/sessions accepts: active_minutes, limit, include_global, include_unknown query params. • Filter implementation in SQL: WHERE updated_at > NOW() - INTERVAL '\$ {activeMinutes} minutes'.
--	---

3.5 Token & Cost Display per Session

OpenClaw's sessions table shows cumulative token counts per session. Alluci has no token counting or cost tracking per session.

 FRONT-END CHANGES • Add token and cost columns to the sessions table: formatted as '12.4 K tokens' and '\$0.031'. • Show a progress bar or colour code the token count against the model's context window size.	 BACK-END REQUIRED • The sessions endpoint must aggregate usage_log entries per session key. • SQL: SELECT session_key, SUM(input_tokens+output_tokens) AS total_tokens, SUM(cost) AS total_cost FROM usage_log GROUP BY session_key. • Store model context_length in the model_pricing table to compute the percentage fill.
--	---

3.6 Delete Session Action

OpenClaw's sessions table has a Delete button per row that removes the session key and its message history. Alluci has no way to delete a session from the UI.

 FRONT-END CHANGES • Add a Delete button (trash icon) to each session table row. • Show a confirmation dialog before deletion. • After deletion, remove the row from local state and, if the deleted session was active, switch to the first remaining session.	 BACK-END REQUIRED • DELETE /api/sessions/{key} → removes session config, message history, and usage_log entries for that key. • Returns 204 No Content on success.
---	---

Date range, daily chart, session table, time series, cost breakdown, CSV export

4.1 Usage / Analytics Panel

OpenClaw has a dedicated Usage tab — the most feature-rich panel in the entire application. It provides date-range filtering, summary stat cards, a daily bar chart (Usage Mosaic), a filterable sessions table with configurable columns, a per-session time-series chart, a message-level log, and CSV export. Alluci has no analytics surface whatsoever.

Status

Missing entirely — no usage tracking, no analytics view, no cost data. Must be built from scratch.

4.1.1 Date Range Picker & Summary Stats

FRONT-END CHANGES

- Add 'usage' to ActiveView and a Usage nav item.
- Create a UsagePanel component with a date range row: startDate input, endDate input, timezone toggle (local/UTC).
- Show 6 summary stat cards: Total Tokens, Total Cost, Input Tokens, Output Tokens, Cache Read, Cache Write.
- Show a warning callout if missingCostEntries > 0.

BACK-END REQUIRED

- GET /api/usage/summary?start=2026-01-01&end=2026-03-04 → {total_tokens, total_cost, input, output, cache_read, cache_write, missing_cost_entries}
- Track usage: when any LLM call completes, record input_tokens, output_tokens, cache_read, cache_write, cost, model, provider, session_key, agent_id, channel.
- usage_log table: id, session_key, model, provider, agent_id, channel, input_tokens, output_tokens, cache_read, cache_write, cost, created_at.

4.1.2 Daily Bar Chart (Usage Mosaic)

FRONT-END CHANGES

- Render a 7-day or 28-day bar chart with one bar per day.
- Each bar is segmented by token type (input, output, cache-read, cache-write) in stacked mode, or total only.
- Clicking a bar sets a single-day date filter; Shift+click adds to a multi-day selection.
- Toggle buttons: Tokens / Cost (Y axis), Total / By Type (stacking mode).

BACK-END REQUIRED

- GET /api/usage/daily?start=...&end=... → [{date, input_tokens, output_tokens, cache_read, cache_write, cost}]
- Group by DATE(created_at) in SQL.



REFERENCE IMPLEMENTATION

```
// Use recharts BarChart
import { BarChart, Bar, XAxis, YAxis, Tooltip } from 'recharts';
<BarChart data={dailyData}>
  <Bar dataKey='input_tokens' stackId='a' />
  <Bar dataKey='output_tokens' stackId='a' />
</BarChart>
```

4.1.3 Sessions Table with Configurable Columns



FRONT-END CHANGES

- Below the chart, render a sessions table with: session key, tokens, cost, messages, tools, errors, duration.
- Allow column visibility to be toggled via a column picker dropdown.
- Table is sortable by clicking column headers (tokens/cost/messages/errors/duration).
- Click a row to select it; Shift-click to multi-select.



BACK-END REQUIRED

- GET /api/usage/sessions? start=...&end=...&limit=1000 → [{session_key, model, provider, channel, agent_id, total_tokens, total_cost, message_count, tool_count, error_count, duration_ms}]
- When result count >= 1000, include limit_reached: true in the response.

4.1.4 Per-Session Time-Series Detail Panel



FRONT-END CHANGES

- When one or more sessions are selected, show a detail panel below the table.
- Detail panel: a line chart showing token/cost per turn or cumulative, with toggle buttons.
- Support cursor-range selection (click-drag on chart to zoom into a turn range).
- Below the chart, show a message log table: timestamp, role, tool name, token count, expandable raw text.



BACK-END REQUIRED

- GET /api/usage/sessions/{key}/timeseries → [{turn, input, output, cost, timestamp}]
- GET /api/usage/sessions/{key}/log? role=assistant&tool=...&q=... → [{role, content_snippet, tool_name, tokens, timestamp}]

4.1.5 Tokenised Query Filter System



FRONT-END CHANGES

- Above the sessions table, add a tokenised query input (like a tag input).
- Supported tokens: session:key, agent:id, channel:name, model:name, provider:name, free text.
- Show autocomplete suggestions as the user types each token.
- Clicking a session-row stat value (e.g. a channel name) adds it as a filter chip automatically.



BACK-END REQUIRED

- The filtering can be done client-side if the session count is under 1000.
- For server-side, the sessions endpoint accepts: session_key, agent_id, channel, model, provider, q (free text) query params.

4.1.6 CSV Export

 FRONT-END CHANGES • Add two export buttons: 'Export Sessions CSV' and 'Export Daily CSV'. • The export applies the current date range and query filters. • Use a client-side CSV builder (no server round-trip needed since data is already in memory).	 BACK-END REQUIRED • Alternatively, add GET /api/usage/export/sessions.csv and /api/usage/export/daily.csv endpoints. • Both accept the same filter params as their JSON counterparts.
 REFERENCE IMPLEMENTATION <pre>// buildSessionsCsv helper const csv = ['key,tokens,cost,messages'].concat(sessions.map(s => [s.session_key, s.total_tokens, s.total_cost, s.message_count].join(','))).join('\n'); const blob = new Blob([csv], {type:'text/csv'}); saveAs(blob, 'sessions.csv');</pre>	

Job list, create/edit form, run history, delivery routing

5.1 Cron Jobs Panel — Full Scheduler UI

OpenClaw has a complete scheduler UI: a status row showing next wake time, a filterable/sortable job list, a create/edit form with 4 collapsible sections (schedule, execution, delivery, advanced), and a run history log per job. Alluci has no scheduler at all.

Status

Missing entirely. The task panel in Alluci is a simple to-do list — it is not a cron scheduler and has no concept of recurrence, agent invocation, or delivery routing.

5.1.1 Status Bar & Job List

 FRONT-END CHANGES • Add 'cron' to ActiveView and a nav item. • Status bar: Cron Enabled/Disabled badge, Total Jobs count, Next Wake relative time. • Job list: each row shows enabled toggle, name, schedule summary, next run time, last status badge (ok/error/skipped), agent ID. • Filters: search text, enabled (all/enabled/disabled), schedule kind (all/interval/cron/date), last status. • Sort by: name, next run, last run, created. Sort direction toggle. • Row actions: Clone, Delete, Run Now (force / due mode).	 BACK-END REQUIRED • GET /api/cron/jobs?q=&enabled=&schedule_kind=&last_status=&sort=next_run&dir=asc → [{id, name, enabled, schedule, next_run_at, last_run_status, agent_id}] • cron_jobs table: id, name, enabled, schedule_kind, schedule_expr, agent_id, model_override, thinking_level, payload_kind, message_text, delivery_channel, delivery_account, delivery_to, delivery_mode, session_key, reset_context, created_at. • POST /api/cron/jobs/:id/run?mode=force → triggers immediate run • POST /api/cron/jobs/:id/clone → duplicates job • DELETE /api/cron/jobs/:id
---	--

5.1.2 Job Create / Edit Form

 FRONT-END CHANGES • Modal or inline form with 4 collapsible sections: • Section 1 — Schedule: kind select (interval/cron-expression/run-at-date), conditional fields per kind, timezone autocomplete, enabled checkbox. • Section 2 — Execution: agent ID (autocomplete), model override (autocomplete), thinking override, payload type (post-message/run-task/none), message/task text. • Section 3 — Delivery: channel autocomplete, account autocomplete, recipient, delivery mode (announce-summary/post-transcript/none). • Section 4 — Advanced: session key override, context reset checkbox. • Save / Cancel buttons. Validation: required fields per schedule kind.	 BACK-END REQUIRED • POST /api/cron/jobs {name, schedule, execution, delivery, advanced} → creates job • PUT /api/cron/jobs/:id → updates job • GET /api/cron/status → {enabled, job_count, next_run_at} • The cron engine runs a setInterval (or APScheduler) tick that evaluates all enabled jobs.
 REFERENCE IMPLEMENTATION <pre>// CronForm.tsx — schedule kind conditional fields const ScheduleFields = ({kind, form, onChange}) => { if (kind === 'interval') return <IntervalFields />; if (kind === 'cron') return <CronExprField />; if (kind === 'date') return <DateTimePicker />; };</pre>	

5.1.3 Run History Log



FRONT-END CHANGES

- Clicking a job row expands a run history log below it (or in a side panel).
- Each run entry: timestamp, status badge (ok/error/skipped), duration, delivery status badge, expandable output text.
- Filters on the run log: scope (this job / all jobs), run status (multi-select), delivery status (multi-select), free text.
- Sort direction toggle (newest first / oldest first).
- A 'Load more' button for pagination.

BACK-END REQUIRED

- GET /api/cron/runs?job_id=&scope=this-job&status=ok&delivery_status=sent&q=&sort=desc&page=1
- cron_runs table: id, job_id, started_at, finished_at, status, delivery_status, log_text.
- Write a cron_runs record after every job execution — success, failure, and skipped.

Live health, per-channel config, multi-account, enable/disable toggle

6.1 Live Channel Status Dashboard

OpenClaw's Channels tab shows a 'Channel Health' card with a raw JSON snapshot of all channel statuses and a relative-time 'Last refreshed' label. Errors per channel are surfaced as coloured callout cards. Alluci's BridgeCenter shows connection state per-adapter (CONNECTED / DISCONNECTED) but has no health snapshot, no error surfacing, and no refresh mechanism — it just reflects what bridgeManager reports locally.



FRONT-END CHANGES

- In BridgeCenter, add a 'Health' section at the bottom that calls GET /api/channels/status.
- Render a small status card per channel: channel name, connection state, last message time, error string (if any), account count.
- Add a Refresh button that re-calls the endpoint and shows a 'Last refreshed N seconds ago' label.
- Show error callouts (amber/red) for channels with non-null error fields.

BACK-END REQUIRED

- GET /api/channels/status → {telegram: {connected, error, account_count, last_message_at}, whatsapp: {...}, ...}
- Each adapter's adapter.health() method must return {connected: bool, error: str|None, account_count: int, last_message_at: ISO}.
- Aggregate all adapter health() calls in a /api/channels/status endpoint.

6.2 Per-Channel Config Expansion

OpenClaw shows a collapsible config section per channel card that contains the relevant credential fields (token, webhook URL, phone number, etc.) plus the enable/disable toggle and a list of linked accounts. Alluci's BridgeCard shows only a Connect/Disconnect button — there is no place to enter or edit credentials from the UI. Users must set credentials via environment variables.

FRONT-END CHANGES • Expand each BridgeCard to include a collapsible config section (toggle open/closed on a 'Configure' button). • Inside the config section, render channel-specific fields. Example for Telegram: Bot Token input (masked), Webhook URL (read-only), Enable toggle. • Add a Save button that posts the credentials to the backend. • For channels that require QR pairing (WhatsApp), show the QR image or a 'Generate QR' button.	BACK-END REQUIRED • GET /api/channels/{id}/config → returns current config (credentials masked, e.g. token: '****1234') • PUT /api/channels/{id}/config {token?, webhook_url?, phone_number?} → saves credentials to vault • POST /api/channels/whatsapp/qr → triggers QR generation, returns base64 PNG • GET /api/channels/whatsapp/status → returns {state: 'qr_pending' 'connected' 'disconnected', qr_data?: string} • Store credentials in VaultManager, not plain config files.
--	---

6.3 WhatsApp QR-Code Pairing UI

OpenClaw renders a live QR code image in the WhatsApp channel card during session initialisation. The QR data streams from the gateway and refreshes automatically every 30 s if not scanned. Alluci shows only a Connect button — there is no QR display, no pairing state machine, and no real WA session management.

FRONT-END CHANGES • Add a WhatsApp-specific pairing sub-view inside the BridgeCard when the WhatsApp adapter is selected. • States: IDLE (show 'Generate QR' button), QR_PENDING (show with base64 QR), CONNECTED (show account info), DISCONNECTED. • Poll GET /api/channels/whatsapp/status every 3 s while in QR_PENDING state. • When status changes to CONNECTED, update the bridge connection state and stop polling. • Show a 30s countdown timer on the QR and refresh automatically when it expires.	BACK-END REQUIRED • The WhatsApp adapter must use whatsapp-web.js Client with LocalAuth for session persistence. • On client.on('qr', qr) → encode to base64 PNG, store in-memory, expose via GET /api/channels/whatsapp/status. • On client.on('ready') → emit a channel.connected WS event. • Persist the session to disk (.wwebjs_auth/) so reconnects do not require re-scanning.
REFERENCE IMPLEMENTATION <pre>// whatsapp_adapter.py (pseudocode - use whatsapp-web.js via subprocess or node bridge) class WhatsAppAdapter: async def start(self): self.client = Client(authStrategy=LocalAuth()) self.client.on('qr', lambda qr: self.set_qr(qr)) self.client.on('ready', lambda: self.set_state('connected')) await self.client.initialize()</pre>	

6.4 Channel Enable / Disable Runtime Toggle

OpenClaw allows each channel to be enabled or disabled independently without restarting the server. The toggle is visible on each channel card. When disabled, the adapter stops listening for messages but retains its config. Alluci loads all adapters on startup and has no runtime toggle.

 FRONT-END CHANGES • Add an enabled toggle (switch/checkbox) to each BridgeCard. • On toggle, call PATCH /api/channels/{id} {enabled: bool}. • When the response confirms the change, update the local connection state.	 BACK-END REQUIRED • PATCH /api/channels/{id} {enabled: bool} → starts or stops the adapter at runtime. • In BridgeManager, add start_adapter(id) and stop_adapter(id) methods. • Persist the enabled flag to config so it survives restarts.
--	---

6.5 Per-Account Management per Channel

OpenClaw tracks multiple accounts per channel (e.g. two Telegram bots in the same gateway) and lists them with per-account actions. Alluci is strictly one-account-per-adapter with no account list UI.

 FRONT-END CHANGES • In the config expansion section, add an 'Accounts' sub-list that fetches GET /api/channels/{id}/accounts. • Each account row: account alias, ID, status, last-seen, disconnect action. • An 'Add Account' button triggers the relevant auth flow for another account on the same channel.	 BACK-END REQUIRED • channel_accounts table: id, channel_id, alias, external_id, status, last_seen_at, credentials (encrypted). • GET /api/channels/{id}/accounts → list • DELETE /api/channels/{id}/accounts/{accountId} → disconnect/remove
--	---

6.6 Nostr Channel — Identity Profile Form

OpenClaw's Nostr channel card has a full identity profile form: display name, about text, picture URL, banner URL, NIP-05 identifier, and LUD-16 Lightning address. Alluci has no Nostr adapter and no such form.

 FRONT-END CHANGES • Add a Nostr entry to the groupedConnections VERUS_IDENTITY group. • In the config expansion section, render Nostr profile fields: display_name, about, picture_url, banner_url, nip05, lud16. • A 'Publish Profile' button sends a NIP-01 kind:0 event to the configured relays.	 BACK-END REQUIRED • Nostr adapter: generate or import an nsec/npub keypair, store in VaultManager. • Implement NIP-01 basic events and NIP-04 encrypted DMs using the nostr-tools library. • Store relay list in config: nostr.relays[].
---	---

6.7 Sorted Channel Cards (Enabled First)

OpenClaw renders channel cards sorted: connected/enabled channels first, then disconnected. The order within each group follows a channelMeta order from the gateway. Alluci renders bridges in hard-coded group order regardless of connection state.

 FRONT-END CHANGES • Sort connections in BridgeCenter: <code>.sort((a,b) => (b.status === 'CONNECTED' ? 1 : 0) - (a.status === 'CONNECTED' ? 1 : 0))</code> • Within connected, sort by last message time descending. • Within disconnected, sort alphabetically by name.	 BACK-END REQUIRED • No back-end changes required — this is a client-side sort.
--	---

6.8 Webhook URL Display

For channels that use webhooks (Telegram, Slack, Discord), OpenClaw displays the computed webhook URL as a read-only, copy-on-click field in the channel config. Alluci shows no webhook URL.

 FRONT-END CHANGES • In the config expansion section, render a read-only webhook URL field for applicable channels. • Include a copy-to-clipboard button next to the URL.	 BACK-END REQUIRED • GET <code>/api/channels/{id}/config</code> → include <code>webhook_url</code> in the response, computed as: <code>{VITE_DAEMON_URL}/webhooks/{channel_id}/{secret}</code> • The webhook URL secret should be a per-channel HMAC token stored in the vault.
---	---

6.9 Sync / Action Buttons for Connected Channels

Alluci already has Sync, Search, and Pulse action buttons on connected bridge cards — this is a strength. However, these actions are fire-and-forget: they call `handleSocialAction/handleEnterpriseAction` but no response is surfaced back in the UI. OpenClaw shows the result of channel actions inline on the card as a collapsible response or status badge. The gap is result display, not the action buttons themselves.

 FRONT-END CHANGES • After a Sync/Search/Pulse action completes, show a result summary below the action button: e.g. 'Synced 14 messages' or 'Found 3 files matching...' • Use a loading spinner while the action is pending, then replace with the result text. • Store actionResults: Record<channelId, string> in local BridgeCenter state.	 BACK-END REQUIRED • Bridge action endpoints should return a result payload: {success: bool, message: str, count?: int} • PATCH /api/bridges/{id}/sync → returns {synced_count, last_message_at} • PATCH /api/bridges/{id}/pulse → returns {delivered: bool}
--	--

6.10 iMessage macOS Platform Restriction

OpenClaw renders the iMessage channel card with a platform restriction callout on non-macOS systems, clearly explaining that iMessage is unavailable. Alluci has no such restriction handling — the iMessage/iwatch bridge cards are shown regardless of platform, potentially confusing users on non-Apple systems.

 FRONT-END CHANGES • In BridgeCard, detect macOS: navigator.platform.startsWith('Mac') navigator.userAgent.includes('Macintosh'). • For iMessage, iwatch, iphone adapters: if non-macOS, show a platform callout ('iMessage is only available on macOS') and disable the Connect button. • On macOS, show the normal card.	 BACK-END REQUIRED • GET /api/system/platform → {os: 'darwin' 'linux' 'win32'} — lets the front-end confirm platform from the server side (more reliable than navigator.platform).
---	--

Agent list, identity, model fallbacks, workspace files, per-agent skills & channels

7.1 Multi-Agent Panel with Sidebar Selector

OpenClaw supports multiple named agents, each with their own model, fallbacks, identity, files, tools, skills, and channel subscriptions. The Agents tab has a sidebar listing all agents; selecting one loads a 6-tab panel. Alluci has a single fixed agent — there is no concept of multiple agents, no agent list, and no per-agent configuration panel.

Status Missing entirely. Alluci has one agent only. All agent-related features below are absent.
--

 FRONT-END CHANGES • Add 'agents' to ActiveView. • Create an AgentsPanel with a two-pane layout: fixed-width sidebar listing agents, main area with tabs. • Sidebar: emoji avatar, display name, agent ID (monospace), default/sub badge. • Agent header: emoji, display name, agent ID. • Six tabs: Overview, Files, Tools, Skills, Channels, Cron.	 BACK-END REQUIRED • GET /api/agents → [{id, display_name, emoji, is_default, model, fallbacks}] • agents table: id, display_name, emoji, avatar_url, banner_url, model, fallbacks, tools_profile, created_at. • Alluci's existing single-agent config can become 'agents[0]' to avoid a breaking change.
---	--

7.2 Agent Overview — Primary Model & Fallbacks

OpenClaw's Agent Overview tab has a text input for the primary model, a comma-separated fallbacks input (parsed into an array), identity fields (display name, emoji, avatar URL, banner URL), a dirty indicator, and Save/Reload buttons. Alluci's equivalent is the API wizard — a multi-step modal — not an always-accessible agent overview.

 FRONT-END CHANGES • Agent Overview tab: primary model text input with autocomplete, fallbacks comma-tag input, identity fields. • Show a 'Unsaved changes' indicator when the form differs from the saved state. • Save button: PUT /api/agents/{id}, Reload button: GET /api/agents/{id} to reset the form.	 BACK-END REQUIRED • PUT /api/agents/{id} {model, fallbacks, display_name, emoji, avatar_url} → updates agent config. • GET /api/agents/{id}/identity → {display_name, emoji, avatar_url, banner_url} — served from the agent's identity file.
--	--

7.3 Workspace File Editor (boot.md, skills.md, context.md, tools.md)

OpenClaw's Agent Files tab shows a file-browser sidebar listing the agent's workspace files (boot.md, skills.md, context.md, tools.md, and any custom files). Selecting a file opens it in a textarea editor with draft state tracking, per-file Save and Reset buttons, and a saving spinner. Alluci has no per-agent file editor in the UI.

 FRONT-END CHANGES • Add a Files tab to the AgentsPanel. • Left sidebar: list of files for this agent (name, last modified). • Right area: a textarea editor for the selected file. • Track draft state per file in a Record<filename, string> local state. • Save button is enabled only when the draft differs from the fetched content. • Show a saving spinner while the PUT is in flight.	 BACK-END REQUIRED • GET /api/agents/{id}/files → [{name, size, modified_at}] • GET /api/agents/{id}/files/{name} → raw file content as text/plain • PUT /api/agents/{id}/files/{name} body: text → saves file • Files live at: ~/.alluci/agents/{id}/{name}.md
--	---

7.4 Per-Agent Tool Profile & Override Editor

OpenClaw's Tools tab for each agent has: a tool profile selector (full/restricted/none/custom), an alsoAllow comma-tag input for additions to the restricted set, a deny comma-tag for block-listing, and a Tool Catalog showing every available tool with its current enabled/disabled state based on the effective profile. Alluci's SoulPreferencesPanel has no tool management.

 FRONT-END CHANGES • Add a Tools tab to AgentsPanel. • Tool profile selector: radio or select with options (full / restricted / none / custom). • alsoAllow tag input: comma-separated tool names to add to the profile. • deny tag input: comma-separated tool names to block. • Tool Catalog: flat list of all tools with resolved enabled/disabled state.	 BACK-END REQUIRED • GET /api/agents/{id}/tools → {profile, also_allow[], deny[], catalog: [{name, description, enabled}]} • PUT /api/agents/{id}/tools {profile, also_allow, deny} → updates tool config. • The catalog endpoint resolves the effective set: base_tools(profile) UNION also_allow MINUS deny.
--	--

7.5 Per-Agent Channel Subscriptions

OpenClaw's Agent Channels tab shows which channels the agent listens on (from the channels snapshot), with a toggle per enabled channel. Each agent can be subscribed to a subset of channels. Alluci has no per-agent channel routing.

 FRONT-END CHANGES • Add a Channels tab to AgentsPanel. • List all enabled channels as toggle rows: channel name, icon, toggle. • When a channel is toggled, call PATCH /api/agents/{id}/channels.	 BACK-END REQUIRED • agent_channel_subscriptions table: agent_id, channel_id, enabled. • PATCH /api/agents/{id}/channels {channel_id, enabled} → updates subscription. • The message router checks subscriptions before dispatching an inbound message to an agent.
--	---

7.6 Per-Agent Cron Job List (Read-Only)

OpenClaw's Agent Cron tab shows a read-only list of cron jobs that target this agent, with schedule summary and last status. Alluci has no such view.

 FRONT-END CHANGES • Add a Cron tab to AgentsPanel showing a read-only list of cron_jobs where agent_id === currentAgent. • Each row: job name, schedule summary, last status badge. Link to the main Cron panel.	 BACK-END REQUIRED • GET /api/cron/jobs?agent_id={id} — already covered by the cron endpoint in Section 5.
---	--

7.7 Agent Identity — Emoji Resolver

OpenClaw uses a `resolveAgentEmoji()` function that checks the agent identity result for a configured emoji, falls back to the first character of the agent ID, and finally to a default glyph. Alluci uses generic placeholder text for agent identities.

 FRONT-END CHANGES • Add a utility <code>resolveAgentEmoji(agentId, identityResult)</code> that returns an emoji or initial. • Use this consistently in the agent sidebar and agent headers.	 BACK-END REQUIRED • No back-end changes — use the emoji field from <code>GET /api/agents</code>.
---	--

7.8 Agent Dirty State & Save / Reload Controls

OpenClaw tracks whether the agent config form has unsaved changes and shows a dirty indicator (•). Save and Reload buttons are enabled/disabled based on dirty state and loading state. Alluci's `SoulPreferencesPanel` has a save mechanism but no dirty tracking or visual indicator.

 FRONT-END CHANGES • Track <code>isDirty</code>: boolean per agent panel by comparing current form state to the last fetched state (deep equality check). • Show a small dot or 'Unsaved changes' label when <code>isDirty</code> is true. • Disable Save when not dirty or when saving is in progress. • Reload button resets form to last fetched state (<code>setForm(fetchedData)</code>) and clears dirty state.	 BACK-END REQUIRED • No back-end changes required.
--	---

Status report, missing deps, install flow, API key mgmt, grouping, bulk actions

8.1 Skill Status Report with Missing Dependencies

OpenClaw's Skills tab shows a full status report for each skill: source badge (managed/workspace/built-in/bundled), enabled/disabled state, missing-bins warning, missing-keys warning, and reason chips. Alluci's `SkillGrid` shows skills with only name, description, capabilities, and an active/off toggle. There is no missing-dependency detection, no

source badge, and no reason display.

 FRONT-END CHANGES • Extend the skill card to show: source badge, missing bins warning, missing API keys warning. • Add a 'Reasons' section below the capabilities that explains why a skill is disabled (e.g. 'Missing: ffmpeg', 'API key required'). • Use a colour-coded status chip: green for active and all deps met, amber for active but missing optional deps, red for disabled.	 BACK-END REQUIRED • GET /api/skills/status → [{name, description, source, emoji, enabled, missing_bins[], missing_keys[], reasons[], install[]}] • For each required_bins entry in the skill manifest, check shutil.which(bin) on the server. • Return a structured reasons[] array: [{'Missing binary': 'ffmpeg'}, {'API key required': 'OPENAI_KEY'}].
---	---

8.2 One-Click Skill Installation

When a skill has install descriptors and missing binaries, OpenClaw shows an Install button. Clicking it triggers an install script in the background and shows a progress indicator. Alluci has no install flow — missing dependencies are simply not surfaced.

 FRONT-END CHANGES • Show an Install button on skill cards where missing_bins is non-empty and install[] is non-empty. • On click, call POST /api/skills/{key}/install and show a spinner on the button. • Subscribe to skill.install.progress WS events to update a progress label. • On completion (skill.install.complete event), re-fetch the skills status to refresh the card.	 BACK-END REQUIRED • POST /api/skills/{key}/install → runs the install script in a subprocess, returns immediately (202 Accepted). • The install script emits progress events via WebSocket: {event:'skill.install.progress', data:{skill_key, step, message}}. • On completion: {event:'skill.install.complete', data:{skill_key, success, error?}}.
--	---

8.3 Per-Skill API Key Input

OpenClaw shows an inline API key input on skill cards that require external credentials. The key is saved securely (returns only isSet: bool in the status report). Alluci's ApiWizard handles all API keys in a single batch, not per-skill.

 FRONT-END CHANGES • On skill cards where the skill manifest has apiKey.required = true, show an API key input row. • Input is masked (type='password'). Show 'Key saved ✓' badge when apiKey.isSet is true. • Save button calls POST /api/skills/{key}/apikey {value}. • Never display the stored key value — only show whether it is set.	 BACK-END REQUIRED • POST /api/skills/{key}/apikey {value} → stores key in VaultManager under key 'skill.{key}.apikey', returns {isSet: true}. • The skill status endpoint calls vault.has_secret('skill.{key}.apikey') and returns apiKey.isSet: bool.
---	---

8.4 Skill Source Grouping

OpenClaw groups skills into four categories: managed (expanded by default), workspace (collapsed), built-in (collapsed), bundled (expanded). Each group is a collapsible section. Alluci shows all skills in a flat grid with no grouping.

 FRONT-END CHANGES • Group skills by their source field: managed / workspace / built-in / bundled. • Render each group as a collapsible <details> element or an accordion section. • Default expanded state: managed and bundled expanded, workspace and built-in collapsed. • Show the count of skills per group in the section header.	 BACK-END REQUIRED • No back-end changes — add a source field to the skill manifest YAML/JSON.
--	--

8.5 Skill Filter Input

OpenClaw's Skills tab has a text filter input that filters by name, description, or source. Alluci has no filter on the SkillGrid.

 FRONT-END CHANGES • Add a filter input above the skill grid. • Filter client-side: include skills whose name or description includes the filter text (case-insensitive). • Clear button appears when the filter is non-empty.	 BACK-END REQUIRED • No back-end changes required.
--	--

8.6 Bulk Skill Actions (Per-Agent: Clear & Disable All)

OpenClaw's per-agent Skills sub-tab has two bulk action buttons: Clear (reset to defaults) and Disable All. Alluci has only per-skill toggle buttons.

 FRONT-END CHANGES • In the per-agent Skills section (Agent → Skills tab), add Clear and Disable All buttons. • Clear calls POST /api/agents/{id}/skills/clear → resets all overrides to the global defaults. • Disable All calls POST /api/agents/{id}/skills/disable-all → sets enabled=false for all skills scoped to this agent. • Confirm before executing either bulk action.	 BACK-END REQUIRED • POST /api/agents/{id}/skills/clear → DELETE FROM agent_skill_overrides WHERE agent_id = {id} • POST /api/agents/{id}/skills/disable-all → UPDATE agent_skill_overrides SET enabled=false WHERE agent_id={id} (insert rows if not exists).
---	--

Pairing approval, device list, token rotation, node binding, capability chips

9.1 Device Pairing Approval Workflow

OpenClaw's Nodes tab shows pending device pairing requests as callout rows with Approve and Reject buttons. New requests appear immediately via WebSocket push. Alluci's WebAuthn implementation is a stub that unconditionally succeeds — there is no pairing request list, no approval UI, and no concept of pending devices.

FRONT-END CHANGES

- Add 'nodes' to ActiveView and a nav item.
- Create a NodesPanel with a Pending Devices section at the top.
- Show pending requests as amber callout rows: deviceId, remotelp, requestedAt, Approve/Reject buttons.
- Show a badge count on the nav item when pending count > 0.
- Listen for devices.pairing-request WS events to add new requests dynamically.

BACK-END REQUIRED

- GET /api/devices/pending → [{request_id, device_id, public_key, remote_ip, requested_at}]
- POST /api/devices/{requestId}/approve → approves, creates entry in paired_devices table.
- POST /api/devices/{requestId}/reject → removes from pending.
- pending_devices table: request_id, device_id, public_key, remote_ip, requested_at.
- paired_devices table: device_id, display_name, public_key, remote_ip, version, paired_at, last_seen_at, revoked.
- Emit WS event devices.pairing-request when a new device sends the hello handshake with an unknown device_id.

9.2 Paired Devices List with Status Chips

OpenClaw's Nodes tab shows all approved devices in a list: display name, nodeId, remotelp, version, paired/unpaired chip, connected/offline chip, capability chips, command chips. Alluci has no paired devices list.

 FRONT-END CHANGES • Below the pending section, add an Approved Devices list. • Each device row: display name, device ID (monospace), remote IP, version, status chips (paired/unpaired, connected/offline), capability chips (capped at 12), command chips (capped at 8). • Actions per row: Rotate Token, Revoke (Unpair).	 BACK-END REQUIRED • GET /api/devices → [{device_id, display_name, remote_ip, version, is_connected, capabilities[], commands[], paired_at, revoked}] • Devices broadcast their capabilities in the hello handshake: capabilities: ['system.run', 'browser.screenshot']. • POST /api/devices/{deviceId}/rotate-token → generates a new auth token for the device. • POST /api/devices/{deviceId}/revoke → marks device as revoked.
---	---

9.3 Exec Approval Policy Editor

OpenClaw's Nodes tab has an Exec Approvals section that lets the admin set a policy per agent: **allow-all**, **deny-all**, or **prompt** (interactive per-call approval). Alluci uses a regex-based whitelist (SEMI_AUTONOMOUS mode) with no per-agent policies and no interactive approval flow.

 FRONT-END CHANGES • Add an Exec Approvals section to the NodesPanel. • Target selector: Gateway or a specific Node. • Agent selector: which agent's exec requests this policy applies to. • Mode select: allow-all / deny-all / prompt. • A callout when no policies are set ('Default: allow all — configure policies to restrict'). • Save button calls PUT /api/exec-policies. • When mode is 'prompt', the daemon must send an exec.approval WS event for each tool call, and the UI must show a modal.	 BACK-END REQUIRED • exec_policies table: id, agent_id, target_type ('gateway' 'node'), target_node_id, mode ('allow-all' 'deny-all' 'prompt'). • GET /api/exec-policies → list • PUT /api/exec-policies → upsert • When the agent executes a shell command and the policy is 'prompt': emit exec.approval WS event, then wait (up to 30s) for exec.allow or exec.deny RPC before proceeding or aborting.
 REFERENCE IMPLEMENTATION <pre>// ExecApprovalModal.tsx useEffect(() => { ws.on('exec.approval', (data) => setExecApproval(data)); }, []); return approval && (<Modal> <pre>{approval.command}</pre> <button onClick={() => ws.send({method:'exec.allow',params:{requestId:approval.id}})}>Allow</button> <button onClick={() => ws.send({method:'exec.deny',params:{requestId:approval.id}})}>Deny</button> </Modal>);</pre>	

9.4 Per-Agent Node Binding

OpenClaw allows each agent's shell command execution to be bound to a specific node (Default, or a named node ID). There is a global default binding and per-agent overrides. Alluci executes all commands locally with no concept of remote nodes.

 FRONT-END CHANGES • In NodesPanel, add a Node Bindings section. • Default binding: a select showing 'Any node' or a specific node ID. • Per-agent bindings: one row per agent with its own node select ('Use default' = inherit). • Save button calls PUT /api/config/node-bindings.	 BACK-END REQUIRED • Add to config: tools.exec.node (global default) and agents[N].tools.exec.node (per-agent override). • In the tool execution path: resolve target node → agents[N].tools.exec.node ?? config.tools.exec.node ?? 'any'. • If target is a specific node, route the exec request over the device WebSocket to that node.
--	--

9.5 Live Nodes List with Capability Metadata

OpenClaw's Nodes panel shows all known remote nodes in a list with their capabilities and commands. Each node announces its capabilities in the presence beacon. Alluci has no node list UI.

 FRONT-END CHANGES • Add a 'Nodes' list section below the bindings section. • Each node: display name, node ID (monospace), remote IP, version, capability chips, command chips. • Refresh button re-fetches the presence list.	 BACK-END REQUIRED • GET /api/system/nodes → [{node_id, display_name, remote_ip, version, capabilities[], commands[], last_seen_at}] • Nodes send a presence beacon every 60 s: POST /api/presence {node_id, capabilities, commands}. • Presence entries expire after 5 minutes.
--	---

Schema-driven form mode, raw JSON mode, hot-apply, field search

10.1 Schema-Driven Config Form

OpenClaw's Config tab renders the entire configuration as a schema-driven form — each field gets the appropriate input widget (text, select, checkbox, number, nested expandable object, array list with add/remove) based on the JSON Schema type. There is a left sidebar listing schema sections and a search input that filters by key path. Alluci

has no config editor in the UI — users edit environment variables directly.

Status

Missing. Alluci reads config from .env and hardcoded defaults. There is no config viewer or editor in the UI.



FRONT-END CHANGES

- Add 'config' to ActiveView and a nav item.
- Create a ConfigPanel with two mode tabs: Form and Raw JSON.
- Form mode: fetch the schema (GET /api/config/schema) and render using a recursive renderNode() function.
- renderNode() maps: string + enum → <select>, string → <input text>, boolean → <checkbox>, number → <input number>, object → nested collapsible section, array → list with add/remove.
- Left sidebar: list of top-level schema sections. Clicking scrolls to and highlights that section.
- Search input at the top of the left sidebar filters fields by key path.

BACK-END REQUIRED

- GET /api/config → current config as JSON
- GET /api/config/schema → JSON Schema document for the config structure
- POST /api/config {config} → saves config to disk
- POST /api/config/apply → hot-reloads without restart
- JSON Schema must be maintained alongside the Pydantic config model.



REFERENCE IMPLEMENTATION

```
// Minimal renderNode for schema-driven form
function renderNode(key, schema, value, onChange) {
  if (schema.enum) return <select value={value} onChange={e => onChange(e.target.value)}>
    {schema.enum.map(o => <option key={o}><o></option>)}</select>;
  if (schema.type === 'boolean') return <input type='checkbox' checked={value}
    onChange={e => onChange(e.target.checked)} />;
  if (schema.type === 'object') return Object.keys(schema.properties).map(k =>
    renderNode(k, schema.properties[k], value?.[k], v => onChange({...value, [k]:v})));
  return <input type='text' value={value||''} onChange={e => onChange(e.target.value)} /
>;
}
```

10.2 Raw JSON Edit Mode

OpenClaw's Config tab has a Raw JSON mode: a full-width textarea showing the entire config as indented JSON. A validation badge shows 'Valid' (green) or 'Invalid' (red) in real time as the user types. Alluci has no raw config editor.



FRONT-END CHANGES

- Add a Raw JSON tab to the ConfigPanel.
- Textarea shows JSON.stringify(config, null, 2).
- A validator badge updates on every keystroke: try { JSON.parse(draft); } catch → 'Invalid'.
- Save is disabled while the JSON is invalid.

BACK-END REQUIRED

- No new back-end changes beyond what Section 10.1 adds.

10.3 Hot-Apply Without Restart

OpenClaw has an Apply button that hot-applies the current config without restarting the gateway (separate from Save which just writes to disk). Alluci requires a full process restart to pick up any config changes.

 FRONT-END CHANGES • Add an Apply button alongside the Save button in the ConfigPanel header. • Apply calls POST /api/config/apply and shows a 'Applying...' spinner, then 'Applied ✓' for 3 s. • Save writes to disk; Apply writes to disk AND reloads running services.	 BACK-END REQUIRED • POST /api/config/apply → calls config.reload() which re-reads the config file and re-initialises mutable services. • Services that support hot-reload: ModelRouter (call .reload()), VaultManager (call .reload()), GuardrailEngine (call .reload()). • Services that cannot hot-reload (BridgeManager) should log a warning and require restart.
---	--

10.4 Config Dirty State Indicator

OpenClaw tracks whether the config form has unsaved changes and shows a dirty indicator. Save and Apply are enabled only when dirty. Alluci has no dirty state tracking in its config surfaces.

 FRONT-END CHANGES • Track configDirty: boolean in local ConfigPanel state. • Set dirty=true when any field changes; clear on Save or successful Apply. • Show 'Unsaved changes' label in the panel header when dirty. • Disable Save/Apply buttons when not dirty. • Warn the user if they navigate away while dirty (beforeunload or navigation interceptor).	 BACK-END REQUIRED • No back-end changes required.
---	--

JSONL tail, level chips, auto-follow, export

11.1 Log Streaming Panel

OpenClaw has a Logs tab that streams and renders the daemon's JSONL log file. Each entry is parsed and displayed as time / level / subsystem / message columns with colour-

coded level chips. Alluci has no log viewer in the UI — logs are only accessible via the terminal running the process.

Status

Missing. No log viewer exists in Alluci's UI.

FRONT-END CHANGES

- Add 'logs' to ActiveView and a nav item.
- Create a LogsPanel that fetches a page of recent log entries on mount.
- Render entries as rows: time (toLocaleTimeString), level chip (colour-coded), subsystem label (monospace), message text.
- Level chips: trace (grey), debug (slate), info (blue), warn (amber), error (red), fatal (dark red) — with checkbox-style toggle to show/hide each level.
- Text filter input above the list; filtering is browser-side.
- Truncation callout when the log is too large ('Showing latest chunk').

BACK-END REQUIRED

- GET /api/logs?levels=info,warn,error&limit=500 → [{time, level, subsystem, message, raw}]
- The daemon writes structured JSONL: {time: ISO, level: 'info', subsystem: 'bridge.telegram', message: 'Connected', ...}
- Use structlog or Python logging with a JSONL formatter.

11.2 Auto-Follow Scroll

OpenClaw's log view auto-scrolls to new entries until the user scrolls up, at which point auto-follow is paused. A checkbox in the header re-enables auto-follow and snaps to the bottom. Alluci has no log streaming UI.

FRONT-END CHANGES

- Track autoFollow: boolean in local LogsPanel state (default: true).
- On each new log entry, if autoFollow is true, scroll the list container to the bottom.
- Add an onScroll handler: if the user scrolls up (scrollTop < scrollHeight - clientHeight - 50), set autoFollow=false.
- Add an 'Auto-follow' checkbox in the panel header. Checking it re-enables and scrolls to bottom.

BACK-END REQUIRED

- No back-end changes required beyond the log endpoint added in 11.1.

11.3 Real-Time Log Push via WebSocket

OpenClaw streams new log entries in real time over the WebSocket connection (logs.entry events), so the user sees entries as they are written without refreshing. Alluci's proposed log endpoint is poll-only.

 FRONT-END CHANGES • Subscribe to logs.entry WS events on mount. • Append new entries to the log list. Cap the in-memory list at 2000 entries (remove oldest). • Show a 'LIVE' indicator badge in the panel header when the WS subscription is active.	 BACK-END REQUIRED • Add a log tail handler in the daemon: whenever the logger writes a line, push a WS event. • Implement using a logging handler that writes to a queue; a background task drains the queue and pushes to all subscribed WS clients. • WS event: {event:'logs.entry', data:{time, level, subsystem, message, raw}}
--	---

11.4 Log Export

OpenClaw exports the currently visible (filtered) log entries as a plain text file with a filename that reflects the filter state. Alluci has no log export.

 FRONT-END CHANGES • Add an Export button to the LogsPanel header. • Export the currently filtered and visible entries as plain text (one raw JSONL line per line). • Filename: logs-{isoDate}-{levelFilter}.txt • Disabled when filtered entry count is 0.	 BACK-END REQUIRED • Alternatively: GET /api/logs/export?levels=error,warn&q=bridge → streams filtered log as text/plain. • Content-Disposition: attachment; filename=logs-2026-03-04.txt
--	---

Live event feed, raw RPC caller, security audit, system health snapshots

12.1 Live Event Log Feed

OpenClaw's Debug tab shows a live rolling feed of all WebSocket events received since the UI connected — event name, timestamp, and formatted payload. This is invaluable for diagnosing unexpected behaviour without tail-ing a log file. Alluci's Audit Ledger shows audit entries but not a raw event feed.

 FRONT-END CHANGES • Add 'debug' to ActiveView and a nav item. • In DebugPanel, maintain an eventLog: EventLogEntry[] (capped at 200 entries) in local state. • Subscribe to ALL WebSocket events (using a wildcard handler) and prepend each to the list. • Each row: event name (monospace), timestamp (toLocaleTimeString), formatted payload (JSON.stringify, truncated to 120 chars). • Auto-prepend new events to the top of the list.	 BACK-END REQUIRED • No new back-end changes — all events already flow through the WebSocket. • The daemon should emit a ws.event for every internal state change at debug level so the event log is complete.
--	--

12.2 Manual RPC Console

OpenClaw's Debug tab has a Manual RPC card: a Method text input and a Params JSON textarea. Clicking Call fires the RPC and shows the result (or error) in a monospace pre block. This is the equivalent of a built-in Postman for the daemon API. Alluci has no equivalent.

 FRONT-END CHANGES • In DebugPanel, add a card with: Method text input (with placeholder 'system.status'), Params JSON textarea (with validation badge), Call button. • On click: POST /api/rpc {method, params} and show the response in a code block. • Show errors in a red callout. • Pre-populate with example methods from a dropdown (e.g. 'system.status', 'system.health', 'sessions.list').	 BACK-END REQUIRED • POST /api/rpc {method: str, params: dict} → dispatches the RPC method and returns the result. • This is a thin wrapper that calls the same handler used by the WebSocket dispatcher. • Restrict to admin-authenticated sessions only.
--	--

12.3 Security Audit Summary

OpenClaw's Debug tab renders a colour-coded security callout derived from the system.status response: red if critical > 0, amber if warn > 0, green if clean. It also suggests the CLI command to run a deep audit. Alluci has no security audit surface in the UI.

 FRONT-END CHANGES • In DebugPanel, add a Security Audit card. • Fetch GET /api/system/status which includes security_audit: {critical, warn, info}. • Render a colour-coded callout: green for clean, amber for warnings, red for critical. • Show the counts and a suggested command: alluci security audit --deep.	 BACK-END REQUIRED • In GET /api/system/status, run a security_audit() function that checks: • - Is MASTER_KEY set and > 32 chars? If not → critical • - Is CORS wildcard (*) allowed in production? → warn • - Are any secrets stored in plain config (not vault)? → warn • - Is WebAuthn stub still returning unconditional success? → critical • Return: {critical: int, warn: int, info: int, findings: [{severity, message}]}
---	---

12.4 System Health Snapshots Panel

OpenClaw's Debug tab shows live system.status (uptime, active sessions, memory, cron count) and system.health (per-subsystem health checks) as formatted JSON blocks. Alluci's daemonStatus is a binary ONLINE/OFFLINE flag — there are no subsystem health checks or memory metrics.

 FRONT-END CHANGES • In DebugPanel, add Status and Health cards with Refresh buttons. • Status card: render {uptime_ms, active_sessions, memory_mb, cron_count, auth_mode} as key-value rows (not raw JSON). • Health card: render a list of subsystems with pass/fail chips: database, vault, model_router, guardrail, bridge_manager.	 BACK-END REQUIRED • GET /api/system/status → {uptime_ms, active_sessions, memory_usage_mb, cron_job_count, auth_mode, security_audit} • GET /api/system/health → {status:'ok' 'degraded', checks: {database:'ok', vault:'ok', model_router:'ok', ...}} • Health checks: database → SELECT 1, vault → vault.ping(), model_router → test LLM connection.
---	---

 FRONT-END CHANGES • In OverviewPanel, add Gateway URL, Session Key, Presence Count, Onboarding, and Locale cards.	 BACK-END REQUIRED • GET /api/system/status → {uptime_ms, active_sessions, memory_usage_mb, cron_job_count, auth_mode, security_audit} • GET /api/system/health → {status:'ok' 'degraded', checks: {database:'ok', vault:'ok', model_router:'ok', ...}} • Health checks: database → SELECT 1, vault → vault.ping(), model_router → test LLM connection.
---	--

Gateway URL config, session key pill, presence count, onboarding, locale

13.1 Gateway URL & Token Configuration in Overview

OpenClaw's Overview tab surfaces the WebSocket gateway URL, auth token, and session key as configurable fields in an 'Access' card. First-time users can change the gateway URL directly from the UI. Alluci configures the daemon URL only via the VITE_DAEMON_URL environment variable — there is no in-app configuration card.



FRONT-END CHANGES

- Add an Overview (or Settings) view with a Gateway Access card.
- Fields: Daemon URL (text input, default from VITE_DAEMON_URL), Auth token (password input), Session key (text input).
- Changes update a UiSettings object persisted in localStorage: {daemonUrl, token, sessionKey, theme}.
- On save, re-initialise the daemon connection with the new URL/token.
- Show a connection status pill (green ONLINE / red OFFLINE) next to the daemon URL.



BACK-END REQUIRED

- No new back-end endpoints required.
- The front-end constructs all API calls using the stored daemonUrl from UiSettings.
- Add a VITE_DEFAULT_DAEMON_URL env var as the fallback when localStorage has no saved URL.

13.2 Active Session Key Pill in Topbar

OpenClaw's topbar shows the current session key as a small pill (monospace, clickable to edit). Clicking opens a small popover that lets the user type a different session key. Alluci's topbar shows only the view title — no session key indicator.



FRONT-END CHANGES

- In SystemHeader, add a session key pill after the brand/identity area.
- Pill shows the first 8 chars of the session key in monospace.
- Clicking opens a small inline input pre-filled with the full key.
- Pressing Enter or blurring saves the new key and re-initialises if changed.



BACK-END REQUIRED

- No back-end changes — session key is a client-side connection parameter.

13.3 Live Presence / Instance Count

OpenClaw's Overview shows a live instance count (number of connected UI clients + nodes in the last 5 minutes) and the sessions count. These update in real time via heartbeat events. Alluci's topbar shows only the HeartbeatIndicator (ONLINE/OFFLINE) with no instance or session counts.



FRONT-END CHANGES

- Add presence count and sessions count to the store: presenceCount: number, sessionCount: number.
- Display them as small badge numbers in the topbar or overview card.
- Update presenceCount on system.heartbeat WS events.



BACK-END REQUIRED

- POST /api/presence {client_id, type} → registers a presence beacon with 5-minute TTL.
- GET /api/presence → {count: int, instances: [{id, type, last_seen_at}]}
- Heartbeat events: {event:'system.heartbeat', data:{presence_count, session_count}}

13.4 Onboarding Wizard on First Launch

OpenClaw activates a guided onboarding flow on first launch (?onboarding=1 or when no saved gateway URL is found). The wizard covers: gateway URL, auth token, session key, and first channel. Alluci starts with a blank connect screen — new users have no guided setup.

 FRONT-END CHANGES • On app startup, check localStorage for 'alluci_setup_complete'. If absent, show the onboarding flow. • Onboarding is a full-screen overlay (not a sidebar panel) with 4 steps: (1) Welcome, (2) Daemon URL & Token, (3) First channel setup, (4) Done. • After completing step 4, set 'alluci_setup_complete' = '1' in localStorage. • A 'Skip' button is available from step 2 onward to exit to the main UI.	 BACK-END REQUIRED • GET /api/system/status is called on step 2 to validate the URL and token. • The wizard can pre-check if the daemon is reachable before proceeding.
---	---

13.5 Locale / Language Selector

OpenClaw supports 5 locales (en, zh-CN, zh-TW, pt-BR, de) and provides a language selector in the Overview settings. All UI strings are run through a t() translation helper. Alluci is English-only with no i18n infrastructure.

 FRONT-END CHANGES • Add a language selector to the overview / settings area. • Store the selected locale in UiSettings (localStorage). • Wrap all user-visible strings in a t() helper that looks up the locale bundle. • Ship at minimum an en bundle; add others as community contributions.	 BACK-END REQUIRED • No back-end changes required for basic i18n. • If server-side responses include user-visible text (e.g. error messages), add an Accept-Language header to requests.
---	--

Master Priority Matrix — All 82 Gaps

Priority: Critical = blocks core functionality or has a security implication. High = materially limits production viability. Medium = significantly improves usability. Low =

quality-of-life.

Abort / Stop Generation button (§1.4)	Chat	Critical	XS	Send AbortController signal + POST / api/chat/abort
Message queue while streaming (§2.3)	Chat	Critical	XS	Client-side queue array, no back-end
Context compaction divider (§2.1)	Chat	Critical	S	compaction.stat us WS event + daemon emits marker
Model fallback banner (§2.2)	Chat	Critical	S	model.fallback WS event from ModelRouter
Session selector in chat header (§1.1)	Sessions	Critical	M	GET /api/sessions + PATCH /api/sessions/{key}
Dedicated sessions panel (§3.1)	Sessions	Critical	M	Full sessions table with token counts
Usage analytics panel (§4.1)	Analytics	Critical	L	usage_log table + summary/daily/sessions endpoints
Cron engine + jobs panel (§5.1)	Scheduling	Critical	L	cron_jobs/ cron_runs tables + scheduler tick
Channel health dashboard (§6.1)	Channels	High	S	GET /api/channels/status aggregating adapter.health()
WhatsApp QR pairing UI (§6.3)	Channels	High	M	whatsapp-web.js + QR stream endpoint
Per-channel config expansion (§6.2)	Channels	High	M	GET/PUT /api/channels/{id}/config
Per-session model override (§3.3)	Sessions	High	S	model col in sessions table, PATCH endpoint
Per-session thinking level (§3.2)	Sessions	High	S	thinking_level col + PATCH endpoint
Jump to new messages button (§1.6)	Chat	High	XS	Client-side scroll detection, no back-end
Focus mode button (§1.3)	Chat	High	XS	Client-side layout state only

Log streaming panel (§11.1)	Observability	High	M	GET /api/logs + logs.entry WS events
Exec approval modal (§9.3)	Devices	High	M	exec.approval WS event + exec_policies table
Device pairing approval UI (§9.1)	Devices	High	M	pending_devices /paired_devices tables + WS event
Multi-agent panel (§7.1)	Agents	High	L	agents table + 6-tab panel
Workspace file editor (§7.3)	Agents	High	M	GET/PUT /api/agents/{id}/files/{name}
Config panel with form mode (§10.1)	Config	High	M	GET /api/config/schema + JSON Schema doc
Schema-driven config raw mode (§10.2)	Config	High	S	Same endpoints as 10.1
Hot-apply config (§10.3)	Config	High	S	POST /api/config/apply + service .reload() methods
Skill status report + missing deps (§8.1)	Skills	High	M	GET /api/skills/status + shutil.which() checks
Per-skill API key input (§8.3)	Skills	High	S	POST /api/skills/{key}/apikey + VaultManager
Daily bar chart (§4.1.2)	Analytics	High	S	GET /api/usage/daily + recharts
CSV export (§4.1.6)	Analytics	Medium	XS	Client-side builder, no back-end needed
Per-agent tool profile editor (§7.4)	Agents	Medium	M	GET/PUT /api/agents/{id}/tools
Per-agent channel subscriptions (§7.5)	Agents	Medium	S	agent_channel_s subscriptions table
Channel enable/disable toggle (§6.4)	Channels	Medium	S	PATCH /api/channels/{id}{enabled}
Per-account management (§6.5)	Channels	Medium	M	channel_account_s table
Node binding editor (§9.4)	Devices	Medium	M	node binding fields in config + routing in exec path

Tokenised query filter (§4.1.5)	Analytics	Medium	S	Client-side filter logic
One-click skill install (§8.2)	Skills	Medium	M	POST /api/skills/{key}/install + progress WS event
Skill source grouping (§8.4)	Skills	Medium	XS	Client-side groupBy, no back-end
Security audit card (§12.3)	Debug	Medium	S	security_audit() in GET /api/system/status
Manual RPC console (§12.2)	Debug	Medium	S	POST /api/rpc passthrough
Live event log feed (§12.1)	Debug	Medium	S	All WS events forwarded to subscribed debug clients
Gateway URL card / overview (§13.1)	Shell	Medium	XS	localStorage UiSettings, no back-end
Session key pill in topbar (§13.2)	Shell	Medium	XS	Client-side only
Auto-follow log scroll (§11.2)	Observability	Medium	XS	Client-side scroll detection
Markdown artifact split pane (§1.5)	Chat	Medium	S	Artifact detection in message stream
Per-session time-series detail (§4.1.4)	Analytics	Medium	M	GET /api/usage/sessions/{key}/timeseries
Cron delivery routing (§5.1.2)	Scheduling	Medium	M	delivery fields in cron_jobs
Cron run history log (§5.1.3)	Scheduling	Medium	S	cron_runs table
System health snapshots (§12.4)	Debug	Medium	S	GET /api/system/health checks
Onboarding wizard (§13.4)	Shell	Low	S	Multi-step overlay, localStorage flag
Log export (§11.4)	Observability	Low	XS	Client-side text download
Per-channel webhook URL display (§6.8)	Channels	Low	XS	Include in GET /api/channels/{id}/config
Sorted bridge cards (§6.7)	Channels	Low	XS	Client-side sort, no back-end
Locale selector (§13.5)	Shell	Low	M	i18n infrastructure + translation bundles
iMessage platform restriction (§6.10)	Channels	Low	XS	Client-side platform detection

Nostr identity profile form (§6.6)	Channels	Low	L	Nostr adapter + NIP-01/04
Animated reading indicator (§2.4)	Chat	Low	XS	CSS keyframe animation, no back-end
Bulk skill actions (§8.6)	Skills	Low	S	POST /api/agents/{id}/skills/clear and disable-all
Hoverable copy icon per message (§1.7)	Chat	Low	XS	CSS :hover, no back-end
Source attribution improvements (§2.5)	Chat	Low	XS	Front-end truncation tweak
Agent dirty state indicator (§7.8)	Agents	Low	XS	Client-side deep-equal comparison
Live presence / instance count (§13.3)	Shell	Low	S	Presence beacon table + heartbeat events

Suggested Sprint Delivery Plan

Sequenced to front-load critical security fixes, foundational plumbing, and high-visibility chat improvements before building out the heavier panels.

Sprint 0 — Security Hotfixes (3 days)

Do first

These are bugs, not features. Fix them before any public usage.

- Replace == with hmac.compare_digest() for all secret comparisons (§ audit BUG-05) — 30 min
- Implement real WebAuthn verification — 1 day (§9.1 pre-requisite)
- Fix NameError in /ready endpoint and duplicate exception handler — 1 hr
- Encrypt BioVault state tokens via VaultManager — 2 hrs

Sprint 1 — Chat & Streaming Polish (1 week)

- Abort/Stop button (§1.4) — XS

- Jump to new messages button (§1.6) — XS
- Focus mode toggle (§1.3) — XS
- Message queue while streaming (§2.3) — XS
- Context compaction divider (§2.1) — S
- Model fallback banner (§2.2) — S
- Animated reading indicator (§2.4) — XS

Sprint 2 — Sessions & Core State (1 week)

- Session selector in chat header (§1.1) — M
- Thinking level toggle in chat (§1.2) — S
- Dedicated sessions panel with table (§3.1) — M
- Per-session model + thinking overrides (§3.2, 3.3) — S
- Session token/cost display (§3.5) — S
- Delete session (§3.6) — S

Sprint 3 — Usage Analytics (1.5 weeks)

- usage_log table + per-turn token tracking — M
- Summary stats + date range picker (§4.1.1) — S
- Daily bar chart (§4.1.2) — S
- Sessions table with configurable columns (§4.1.3) — M
- CSV export (§4.1.6) — XS

Sprint 4 — Config + Debug + Logs (1 week)

- Config panel with form + raw JSON mode (§10.1, 10.2) — M
- Hot-apply config (§10.3) — S
- Log streaming panel + auto-follow (§11.1, 11.2) — M
- Log push via WebSocket (§11.3) — S
- Debug panel: event log, RPC console, security audit, health (§12.1–12.4) — M

Sprint 5 — Channels & Bridges (2 weeks)

- Channel health dashboard (§6.1) — S
- Per-channel config expansion (§6.2) — M
- WhatsApp QR pairing UI (§6.3) — M
- Channel enable/disable toggle (§6.4) — S

- Webhook URL display (§6.8) — XS
- Sorted bridge cards (§6.7) — XS
- iMessage platform restriction (§6.10) — XS

Sprint 6 — Scheduling (2 weeks)

- Cron engine + job list + status bar (§5.1.1) — L
- Job create/edit form (§5.1.2) — M
- Cron delivery routing — M
- Run history log (§5.1.3) — S

Sprint 7 — Agents & Skills (2 weeks)

- Multi-agent panel + sidebar (§7.1) — L
- Agent overview: model, fallbacks, identity (§7.2) — S
- Workspace file editor (§7.3) — M
- Per-agent tool profile editor (§7.4) — M
- Per-agent channel subscriptions (§7.5) — S
- Skill status report + deps + install + API keys (§8.1–8.3) — M
- Skill grouping + filter (§8.4, 8.5) — XS

Sprint 8 — Devices, Nodes & Shell Polish (1.5 weeks)

- Device pairing approval UI (§9.1, 9.2) — M
- Exec approval modal (§9.3) — M
- Per-agent node binding (§9.4) — M
- Gateway URL card + session key pill (§13.1, 13.2) — XS
- Onboarding wizard (§13.4) — S
- Presence / instance count (§13.3) — S
- Per-session time-series detail (§4.1.4) — M
- Markdown artifact split pane (§1.5) — S

Sprint 9+ — Lower Priority Items

- Nostr channel adapter + identity form (§6.6) — L
- Per-account management per channel (§6.5) — M
- Locale selector + i18n infrastructure (§13.5) — M
- Per-agent cron read-only list (§7.6) — S

- Bulk skill actions (§8.6) — S
- Agent dirty state indicator (§7.8) — XS
- Log export (§11.4) — XS